画像編集ツール 担当A 詳細設計書

作成日	2026年10月19日 11:50
科目名	プログラミング言語

参照：詳細設計、AG-01〜AG-09 担当A回答、概要設計、開発分担・進行計画

> 🧩 本書はVersion 1における**担当A（エディタ本体・UI・状態管理・統合）**の実装詳細を定義する。担当Bの詳細設計とAG-01〜AG-09の合意内容を接続点の正本とし、画像処理アルゴリズム、AI推論、PSD解析内部はB側設計を参照する。本書で新たに具体化した項目はA側実装詳細として固定し、共通contract・HTTP API・DTO・ビルド設定・履歴方式・`.jimg`スキーマを変更する場合は実装前に相互確認する。

1. 担当範囲と完了条件
    1.1 Aの責務
    1.2 A単体の完了条件
    1.3 共同受入条件
    1.4 A側対象外
2. モジュールと依存関係
    2.1 リポジトリ
    2.2 A側パッケージ
    2.3 呼び出し方向
    2.4 禁止依存
3. アプリケーションライフサイクルとEditorSession
    3.1 `MainApplication`
    3.2 `AppContext`
    3.3 `EditorSession`
4. Documentモデルと世代管理
    4.1 `ImageDocument`
    4.2 `contentRevision`

- 4.3 Document切替
- 5. Layer・LayerManager・描画
  - 5.1 `Layer`
  - 5.2 `RasterLayer`
  - 5.3 `TextLayer`
  - 5.4 `DrawingLayer`
  - 5.5 `LayerManager`
  - 5.6 `DocumentRenderer`
- 6. UI構成とController責務
  - 6.1 メイン画面
  - 6.2 Controller
  - 6.3 UIモード
  - 6.4 ショートカット
- 7. Tool入力と座標変換
  - 7.1 共通 `Tool`
  - 7.2 座標系
  - 7.3 `SelectTool`
  - 7.4 `BrushTool`
  - 7.5 `EraserTool`
  - 7.6 `CropTool`
- 8. Layer単体・Document全体の幾何編集
  - 8.1 Layer単体
  - 8.2 Document画像サイズ変更
  - 8.3 Canvasサイズ変更
  - 8.4 Canvas Crop
  - 8.5 Document全体90度回転
- 9. Undo / Redoと履歴メモリ
  - 9.1 `Command`
  - 9.2 `HistoryManager`
  - 9.3 Command種別
  - 9.4 `TileDiff`

9.5 256MiB上限
9.6 Redo失敗
10. ファイルI/Oと `.jimg` Version 1
10.1 PNG/JPEG読込
10.2 書き出し
10.3 `.jimg` 構造
10.4 保存安全性
10.5 読込検証
11. Preview・非同期処理・エラー表示
11.1 Preview
11.2 `previewRequestId`
11.3 `ProcessingCoordinator`
11.4 ErrorCode表示
12. AI選択UIと生成Layer
12.1 A側状態
12.2 開始
12.3 クリック
12.4 候補表示
12.5 確定
12.6 失効条件
13. PSD DTOの内部Documentへの登録
13.1 `PsdImportCoordinator`
13.2 Node変換
13.3 Metadata
13.4 Warning UI
14. A側テスト設計
14.1 共同結合テスト
15. 実装順・結合運用・7週間成果物
15.1 7週間のA成果物
15.2 Git運用
15.3 定例結合

15.4 遅延時
16. 要件・合意事項との対応と実装記録
16.1 実装時に残す記録

1. 担当範囲と完了条件

1.1 Aの責務

対象	優先度	Aの責務	Bとの境界
FR-01〜04	MUST / SHOULD	PNG/JPEG入出力、未保存管理、`.jimg`保存・再読込	通常画像I/OはA単独
FR-08〜10	MUST	Text、Brush、Eraserの入力・Document反映・履歴化	BのProcessorを必要としない対話編集はA
FR-16〜24	MUST / SHOULD	Layer追加・削除・選択・名称・表示・順序・移動・透明度・変形	BはLayerManagerへ触らない
FR-05〜07 / 11〜15	MUST / SHOULD	対象決定、入力値、Preview、B処理結果の検証・反映・履歴化	画素演算はB
FR-29〜34	MUST	AI操作UI、座標逆変換、最大3候補表示、候補選択、生成Layer、Undo/Redo	登録・推論・マスク返却はB
FR-25〜27	SHOULD	PSD DTOを内部Layerへ変換、metadata保持、警告表示	PSD解析・中間DTO生成はB
FR-35〜38	MUST / SHOULD	Undo/Redo、Zoom、Pan、UI状態	履歴スタックはA所有

1.2 A単体の完了条件

- PNG/JPEGを開き、DocumentとLayerを生成して編集・保存できる。
- Layerの追加・削除・移動・順序・表示・透明度・名称変更をCommand経由で実行できる。
- Text、Brush、Eraserを操作でき、1操作単位でUndo/Redoできる。
- BのStubを使い、成功・失敗・取消・古い結果をUI側で正しく処理できる。

- AI候補を最大3件表示し、最高score以外も選択でき、確定時だけ新規RasterLayerを生成できる。
- `.jimg`を保存・再読込して主要編集状態を復元できる。
- 失敗・取消・PreviewではDocument、dirty、Historyを不正に変更しない。

1.3 共同受入条件

画像を開く → Bの基本処理 → Aの表示・履歴登録 → AI候補確認 → Aの新規Layer生成 → 移動 → Undo/Redo → PNG/JPEG書き出し、を一続きで確認する。PSDはMUSTとは分離してSHOULDとして評価する。

1.4 A側対象外

- MobileSAM・psd-tools内部実装
- 画像処理アルゴリズムの実装
- Pythonサービス内部のキュー、キャッシュ、デバイス切替
- PSD完全互換・PSD再書き出し
- AI画像生成・クラウド送信
- MUST安定前のADVANCED機能

---

2. モジュールと依存関係

2.1 リポジトリ

```
root/
├─ editor-app/          # A
├─ common-contract/     # A+B共同
├─ processing-backend/  # B
├─ python-service/      # B
├─ gradle/
└─ settings.gradle.kts
```

2.2 A側パッケージ

```
jp.ac.example.imageeditor
├─ app
│  ├─ MainApplication
```

```
│  ├─ AppContext
│  └─ EditorSession
├─ ui
│  ├─ MainWindowController
│  ├─ CanvasController
│  ├─ LayerPanelController
│  ├─ ToolBarController
│  ├─ PropertyPanelController
│  ├─ StatusBarController
│  └─ DialogService
├─ document
│  ├─ ImageDocument
│  ├─ DocumentService
│  ├─ DocumentRenderer
│  ├─ CanvasState
│  └─ RevisionGenerator
├─ layer
│  ├─ Layer
│  ├─ RasterLayer
│  ├─ TextLayer
│  ├─ DrawingLayer
│  ├─ LayerManager
│  ├─ PsdOrigin
│  └─ PsdImportMetadata
├─ tool
│  ├─ Tool
│  ├─ ToolContext
│  ├─ SelectTool
│  ├─ CropTool
│  ├─ BrushTool
│  ├─ EraserTool
│  ├─ TextTool
│  └─ AiSelectTool
├─ history
│  ├─ Command
│  ├─ HistoryManager
│  ├─ TileDiff
│  ├─ RasterEditCommand
│  ├─ AddLayerCommand
│  ├─ DeleteLayerCommand
│  ├─ LayerPropertyCommand
│  └─ DocumentTransformCommand
├─ file
│  ├─ ImageFileService
│  ├─ ExportService
│  ├─ ProjectFileService
│  └─ ProjectSchemaV1
└─ integration
   ├─ ProcessingCoordinator
   ├─ AiSelectionCoordinator
   ├─ PsdImportCoordinator
   └─ ErrorPresenter
```

`common-contract` にはAG-01で合意した `contract.processing`、`contract.ai`、`contract.psd` を置く。

2.3 呼び出し方向

```
JavaFX Controller
    -> A Coordinator / DocumentService
       -> common-contract interfaces
          -> B Processor / AI / PSD implementation
       <- Result / typed exception
    -> A validates target and revision
    -> Command -> Document / LayerManager
    -> Renderer -> UI
```

2.4 禁止依存

- ControllerからBの具体実装クラスを直接newしない。
- UIからLayerの内部ListやBufferedImageを直接書き換えない。
- AからMobileSAM・psd-tools・Python内部モジュールへ直接依存しない。
- Bの結果を受け取ったCoordinatorがHistoryを経由せずDocumentへ破壊的変更しない。
- `common-contract` にJavaFX型、A所有のDocument/Layer型を入れない。

---

3. アプリケーションライフサイクルとEditorSession

3.1 `MainApplication`

```
class MainApplication extends Application {
    void start(Stage stage);
    void stop();
}
```

起動順序：

1. 設定読込
2. `AppContext` 生成

3. JavaFXメイン画面生成
4. `LocalServiceManager.start()`呼出し
5. サービス準備状態をStatusBarへ反映

終了順序：

1. 未保存Documentがあれば保存／破棄／取消を確認
2. バックグラウンド処理へ取消要求
3. `LocalServiceManager.close()`
4. UI終了

AI初期化失敗でもアプリ本体は起動し、通常編集を継続する。

3.2 `AppContext`

アプリ全体で1インスタンスとし、Controllerへ依存を注入する。

```
record AppContext(
    DocumentService documents,
    HistoryManager history,
    ProcessingCoordinator processing,
    AiSelectionCoordinator ai,
    PsdImportCoordinator psd,
    ImageFileService imageFiles,
    ProjectFileService projects,
    DialogService dialogs
) {}
```

3.3 `EditorSession`

UI一時状態をDocumentの永続状態から分離する。

```
class EditorSession {
    UUID activeDocumentId;
    Tool activeTool;
    double zoom;
    double panX;
    double panY;
    EditorMode mode;
```

```
    Optional<AsyncOperation<?>> activeOperation;
}
```

`zoom`、`pan`、一時選択矩形、AI候補Previewは原則Document保存対象外とする。

4. Documentモデルと世代管理

4.1 `ImageDocument`

```
class ImageDocument {
    UUID id;
    int canvasWidth;
    int canvasHeight;
    LayerManager layerManager;
    UUID activeLayerId;
    Path sourcePath;
    Path projectPath;
    long revisionCounter;
    long savedStateId;
    Map<UUID, PsdImportMetadata> psdImports;
}
```

主要操作：

```
Layer getActiveLayer();
void setActiveLayer(UUID layerId);
long nextRevision();
BufferedImage renderComposite();
boolean hasUnsavedChanges();
```

`dirty`は単純な手動フラグではなく、Historyの現在状態IDと`savePointStateId`の一致で判定する。Preview、Zoom、Pan、候補切替では状態IDを進めない。

4.2 `contentRevision`

各`RasterLayer`はRaster画素の世代を示す`long contentRevision`を保持する。

- Raster画素が確定変更されたときだけ`ImageDocument.nextRevision()`で新番号を発行する。
- Undo/Redoでも過去番号を再利用せず、新しい番号を発行する。
- 位置、回転、拡縮、opacity、visible、name、順序変更ではRaster画素が変わらないため`contentRevision`を維持する。
- Bへ渡す`sourceRevision`は`documentId + ":" + contentRevision`とし、Document間の衝突を避ける。

4.3 Document切替

Document切替時は、前DocumentのPreviewとAIセッションを失効させる。完了待ちのB結果が返っても、対象Document IDが現在の操作コンテキストと一致しなければ適用しない。

---

5. Layer・LayerManager・描画

5.1 `Layer`

```
abstract class Layer {
    UUID id;
    String name;
    boolean visible;
    double opacity;       // 0.0..1.0
    double x;
    double y;
    double rotationDeg;
    double scaleX;
    double scaleY;
    PsdOrigin psdOrigin;  // nullable
}
```

規則：

- 座標原点はCanvas左上。
- `x`, `y`は未変形Layerの左上位置。
- `scaleX`, `scaleY`は正値のみ。

- 回転・拡縮のpivotは**Layerの未変形矩形中心**とする。これはA側の座標変換・HitTest・AI overlayで共通使用する。

5.2 `RasterLayer`

```
class RasterLayer extends Layer {
    BufferedImage image;        // TYPE_INT_ARGB_PRE
    long contentRevision;
}
```

確定済み`image`は原則置き換え式とし、Bへ渡したスナップショットを処理中に変更しない。

5.3 `TextLayer`

```
class TextLayer extends Layer {
    String text;
    String fontFamily;
    double fontSize;
    int argb;
    boolean bold;
    boolean italic;
}
```

Version 1ではテキスト情報を保持したまま描画し、PSD由来TypeLayerは再編集可能TextLayerへ変換せずBのDTOどおりRasterLayerとして扱う。

5.4 `DrawingLayer`

```
class DrawingLayer extends Layer {
    BufferedImage image;
    long contentRevision;
}
```

Version 1ではBrush結果をRasterとして保持する。Vector Strokeの永続化は対象外とする。

5.5 `LayerManager`

```
void addLayer(Layer layer);
void insertLayer(int index, Layer layer);
Layer removeLayer(UUID id);
void moveLayer(UUID id, int newIndex);
void setVisible(UUID id, boolean visible);
void setOpacity(UUID id, double opacity);
void rename(UUID id, String name);
void translate(UUID id, double dx, double dy);
Optional<Layer> find(UUID id);
List<Layer> getLayersReadOnly();
```

- index 0を最背面、末尾を最前面とする。
- `getLayersReadOnly()`は変更不可ビューを返す。
- activeLayer削除時は、削除位置の直下、なければ直上、どちらもなければnullを選択する。
- ControllerはLayerManagerを直接変更せず、DocumentServiceまたはCommand経由で変更する。

5.6 `DocumentRenderer`

描画順：

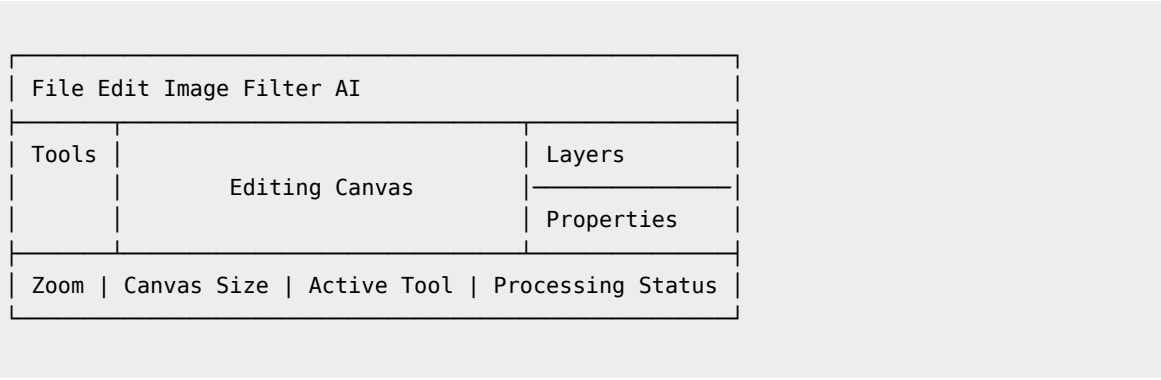
1. 透明Canvasを`TYPE_INT_ARGB_PRE`で作成
2. Layerをindex 0から昇順に描画
3. invisibleはskip
4. Layer transform（位置→中心pivotの回転・scale）を適用
5. Layer opacityを1回だけ適用
6. Canvas外をclip

透明市松模様はUIだけに表示し、Documentの合成画像やPNGへ焼き込まない。

---

6. UI構成とController責務

6.1 メイン画面



6.2 Controller

Controller	責務	持たせないもの
MainWindowController	Menu、Document切替、全体状態	画素演算
CanvasController	描画、overlay、mouse event、Zoom/Pan	Layer Listの直接変更
LayerPanelController	Layer一覧・選択・順序・表示・名称	History stack操作
PropertyPanelController	位置・opacity・Text属性等の入力	B Processor内部呼出し
ToolBarController	Tool切替	Toolロジック
StatusBarController	Zoom、寸法、Service/処理状態	Document変更

6.3 UIモード

```
enum EditorMode {
    IDLE,
    TOOL_GESTURE,
    PREVIEW_DIALOG,
    AI_SELECT,
    DOCUMENT_BUSY
}
```

`DOCUMENT_BUSY` では対象Documentへの破壊的編集を無効化するが、表示、Zoom/Pan、別Document操作は継続可能とする。

6.4 ショートカット

- Cmd+O：開く
- Cmd+S：Project保存。Project未作成時は保存先選択

- Cmd+Shift+S：別名Project保存
- Cmd+Z：Undo
- Cmd+Shift+Z：Redo
- Cmd++ / Cmd+-：Zoom
- Escape：現在Tool/Preview/AI操作の取消

---

7. Tool入力と座標変換

7.1 共通 `Tool`

```
interface Tool {
    void onMousePressed(ToolContext ctx, double x, double
y, MouseButton button);
    void onMouseDragged(ToolContext ctx, double x, double
y, MouseButton button);
    void onMouseReleased(ToolContext ctx, double x, double
y, MouseButton button);
    void cancel(ToolContext ctx);
}
```

1 gestureを原則1 Commandとし、drag中の各イベントを履歴へ積まない。

7.2 座標系

- Screen座標：JavaFX Node内座標
- Canvas座標：原寸Canvas基準
- Layer座標：Layer画像／Textのローカル座標

Screen → Canvas：Panを引き、Zoomで割る。Canvas → Layer：Layer位置を引いた後、中心pivotを基準に逆回転・逆scaleする。Raster座標は最後に`floor`して整数画素へ変換し、`0 <= x < width`、`0 <= y < height`を満たす場合だけ有効とする。

7.3 `SelectTool`

- Click：最前面からHitTestしactiveLayerを変更。
- Drag：LayerをPreview移動し、Release時に`MoveLayerCommand`1件を確定。

- Escape：開始位置へ戻し、Historyを変更しない。

7.4 `BrushTool`

- activeLayerがDrawingLayerならそこへ描画する。
- それ以外では最初のStroke開始時に透明DrawingLayerをactiveLayer直上へ作成し、StrokeとLayer追加を1つの複合Commandとして扱う。
- 1 Stroke = 1 Undo単位。
- 初回接触したTileのbefore画素を保存し、Release時に差分を確定する。

7.5 `EraserTool`

- RasterLayerまたはDrawingLayerのみ対象。
- TextLayerは直接消去せず、必要な場合は明示的なRaster化後に使用する。
- alphaを0へ近づけるRaster編集として扱い、1 Stroke = 1 Command。

7.6 `CropTool`

Toolは矩形選択のみ担当し、確定時に対象種別を明示する。

- Layer Crop：B の `CROP` へ画像内矩形を渡す。
- Canvas Crop：A側Document操作として8章の規則を適用する。

---

8. Layer単体・Document全体の幾何編集

AG-04に基づき、同じ「Resize/Crop/回転」でもLayer単体とDocument全体を別操作として扱う。

8.1 Layer単体

- Raster/DrawingのResize、Crop、90度回転、反転はB Processorへ1枚の画像を渡す。
- 成功結果受領後に対象Layer・`contentRevision` が一致することを再確認する。
- 適用時に新しい `contentRevision` を発行する。
- Crop時は切り取った画像内容に合わせてLayer位置を `x += cropX * scaleX` 、 `y += cropY * scaleY` を基本とする。回転中Layer等で表示位置補正が必要な場合は同一Transform計算を用いて見た目の基準点を維持する。

8.2 Document画像サイズ変更

倍率`Sx = newWidth / oldWidth`、`Sy = newHeight / oldHeight`を求める。

- Canvas：新しい幅・高さへ変更。
- Raster/Drawing：各Layer画像をB Resizeで`round(oldPixelWidth*Sx)` × `round(oldPixelHeight*Sy)`へ変更し、`x *= Sx`、`y *= Sy`。既存rotationは維持する。
- Text：`x *= Sx`、`y *= Sy`、`fontSize *= Sy`、`scaleX *= Sx/Sy`とし、非等倍でも編集可能性を維持する。

A側詳細化上の注意：この方式は各Layerの編集可能性を維持することを優先し、回転済みLayerを含む非等倍Resizeは「全体をFlattenしてCanvas座標系で非等倍変形」した結果と完全一致しない。Version 1ではこの差を仕様とし、Flattenは自動で行わない。

処理は全対象Layerの結果を先に準備し、1件でも失敗・取消があればDocumentへ何も適用しない。成功時のみ1つの`DocumentTransformCommand`として確定する。

8.3 Canvasサイズ変更

画素・Layer寸法を変更せず、Canvasサイズと原点オフセットだけを変更する。Layerの見切れ部分も削除しない。

8.4 Canvas Crop

新Canvasを選択矩形サイズへ変更し、全Layerの`x -= cropX`、`y -= cropY`を適用する。完全にCanvas外となるLayerも保持し、描画時にclipする。

8.5 Document全体90度回転

旧Canvasを`W x H`、Layer中心を`(cx, cy)`とする。

- 右90度：新Canvas=`H x W`、新中心=`(H-cy, cx)`、`rotationDeg += 90`
- 左90度：新Canvas=`H x W`、新中心=`(cy, W-cx)`、`rotationDeg -= 90`

各Layerの新`x/y`は新中心から未変形矩形中心を戻して算出する。TextLayer等はRaster化しない。Document全体を1 CommandでUndoする。

---

9. Undo / Redoと履歴メモリ

9.1 `Command`

```
interface Command {
    void execute() throws CommandException;
    void undo() throws CommandException;
    void redo() throws CommandException;
    long retainedBytes();
    String label();
}
```

9.2 `HistoryManager`

```
void execute(Command command);
boolean canUndo();
boolean canRedo();
void undo();
void redo();
void markSaved();
boolean isDirty();
long usedBytes();
```

各編集状態に一意なstate IDを割り当て、保存時のstate IDと現在state IDが一致する場合だけ`dirty=false`とする。これにより保存後にUndoして保存状態へ戻った場合も正しく判定できる。

9.3 Command種別

- `AddLayerCommand`
- `DeleteLayerCommand`
- `MoveLayerCommand`
- `ReorderLayerCommand`
- `LayerPropertyCommand`
- `TextEditCommand`
- `RasterEditCommand`
- `ReplaceRasterCommand`
- `DocumentTransformCommand`

- `ImportPsdCommand`
- `CompositeCommand`

9.4 `TileDiff`

```
record TileDiff(
    int x, int y, int width, int height,
    int[] beforeArgb
) {}
```

- 1Tile最大256×256px。
- B の `sourceChangedBounds` をTile境界へ拡張し、実画素差があるTileだけ保持する。
- 同サイズで決定的な処理は `beforeArgb + operation + parameters + algorithmVersion` を保持し、Redo時に同じB Processorを再実行できる。
- AI生成LayerはRedoで再推論せず、確定済みRasterLayerを再追加する。
- Brush/Eraserはafterを再計算できない場合、必要なafter Tileも保持できる。

9.5 256MiB上限

- `usedBytes` は保持している画素配列・Raster画像等の実データ量を計上する。
- 新Command確定後に上限を超える場合、最古のUndo Commandから破棄する。
- 1操作単体が256MiBを超える場合はDocumentへ適用せず、Version 1では「Undoなしで強制適用」を提供しない。
- 新規編集時はRedo stackを破棄し、その保持メモリも解放する。

9.6 Redo失敗

Redoの計算が必要なCommandでは、成功するまでredoStackからCommandを移動しない。失敗時はDocument、dirty、Historyを変更せず、利用者へエラーを表示する。

---

10. ファイルI/Oと `.jimg` Version 1

10.1 PNG/JPEG読込

`ImageIO`で読込み、必ず独立した`TYPE_INT_ARGB_PRE`へ正規化する。新規Documentは画像1枚のRasterLayerから開始する。

10.2 書き出し

- PNG：Document合成結果のalphaを保持して出力。
- JPEG：alphaを持てないため、Version 1では白背景へ合成してRGBで出力する。
- 書き出しはProject保存とは別操作とし、PNG/JPEG export成功だけでは`.jimg`のsave pointを更新しない。

10.3 `.jimg`構造

```
project.jimg
├─ project.json
├─ layers/
│  ├─ {layerId}.png
│  └─ ...
└─ imports/
   └─ {importId}.json
```

`project.json`例：

```
{
  "format": "JIMG",
  "version": 1,
  "canvas": {"width": 1920, "height": 1080},
  "activeLayerId": "uuid",
  "layers": [
    {
      "id": "uuid",
      "type": "RASTER",
      "name": "Layer 1",
      "visible": true,
      "opacity": 1.0,
      "x": 0.0,
      "y": 0.0,
      "rotationDeg": 0.0,
      "scaleX": 1.0,
```

```
      "scaleY": 1.0,
      "asset": "layers/uuid.png",
      "psdOrigin": null
    }
  ],
  "psdImports": ["imports/import-uuid.json"]
}
```

- RasterLayer/DrawingLayerの画素はPNGとして保存する。
- TextLayerはtext、fontFamily、fontSize、argb、bold、italicをJSONへ保存し、PNGへ固定しない。
- PSD元ファイルそのものは埋め込まず、`imports/{importId}.json`へ元階層・属性・warningを保持する。
- `contentRevision`は実行時競合制御用のため、読込後に新しいRevision系列を割り当ててよい。永続IDとして扱わない。

10.4 保存安全性

A側実装詳細として、同一ディレクトリへ一時ファイルを書き、ZIP作成・検証成功後に置換する。可能なら`ATOMIC_MOVE`を使用し、未対応環境では通常のreplaceへフォールバックする。保存失敗時は既存Projectを残し、save pointを更新しない。

10.5 読込検証

- `format != JIMG`は拒否。
- `version > 1`は「新しいVersionで作成されたProject」として拒否。
- 重複Layer ID、存在しないasset、Canvas/画像寸法不正、ZIP traversalを拒否。
- 途中までDocumentへ反映せず、全検証成功後に新Documentとして公開する。

---

11. Preview・非同期処理・エラー表示

11.1 Preview

色調整等のDialog開始時に原寸スナップショットを固定する。Slider変更ごとにその開始画像からB Processorへ処理要求し、前回Previewを入力にしない。

内部値：

- 明るさ `-255..255` 既定0
- Contrast `0.0..3.0` 既定1.0
- Saturation `0.0..3.0` 既定1.0
- Blur `0..20` 既定3
- Mosaic `1..128` 既定12

UIは明るさ・Contrast・Saturationを`-100..100`で表示し、AG-06の式で内部値へ変換する。

11.2 `previewRequestId`

各Preview要求に単調更新IDを付ける。結果適用直前に最新IDと一致しない結果は無通知で破棄する。Cancel時は進行中要求へ取消を伝え、Document、dirty、Historyを変更しない。

11.3 `ProcessingCoordinator`

UI threadで以下を固定する。

- documentId
- layerId
- contentRevision
- parameters
- 入力スナップショット

B処理完了後、UI threadで再度対象存在・Revision・取消状態を検証し、成功した結果だけをCommandへ変換する。

11.4 ErrorCode表示

Bの`ErrorCode`をAで日本語UIへ変換する。

Code	A側動作
NO_RESULT	AI画面内で追加クリックを促す。通常Error Dialogは出さない
BUSY	処理中表示＋再試行操作
TIMEOUT	再試行を提示

Code	A側動作
MODEL_INIT_FAILED / MODEL_NOT_READY	AIを利用不可表示。通常編集は継続
IMAGE_NOT_FOUND	AI画像を再登録
REVISION_MISMATCH	古いAIセッションを終了し、最新画像から再開
RESOURCE_LIMIT	画像・履歴等の上限理由を表示
INVALID_RESPONSE / 未知Code	汎用Error。詳細は診断ログ
CANCELLED / 旧結果破棄	通常Dialogを出さない

文字列解析で分岐せず、必ず機械判定可能なCodeを使用する。

12. AI選択UIと生成Layer

12.1 A側状態

```
enum AiSelectionState {
    IDLE,
    REGISTERING,
    READY,
    REQUESTING,
    CANDIDATES,
    CONFIRMING
}
```

保持値：

```
documentId
layerId
contentRevision
aiSessionId
serviceInstanceId
imageId
latestRequestId
points[]
candidates[]
selectedCandidateId
```

12.2 開始

1. activeLayerがRasterLayerであることを確認。
2. 元RGBAの固定snapshotを保持。
3. `aiSessionId`を新規生成。
4. BへregisterImage。
5. 完了時にDocument/Layer/contentRevision/sessionを再検証。
6. 失効済みなら返却imageIdをreleaseして終了。

12.3 クリック

- Screen→Canvas→Layer座標へ逆変換し、`floor`して画素座標化。
- 画像外は要求しない。
- 正点／負点を`points`へ追加し、毎回クリック列全体を送る。
- 新しい`requestId`を発行し`latestRequestId`を更新。
- 候補取得中に追加クリックした場合、旧結果は表示しない。

12.4 候補表示

- 1〜3候補を候補切替UIへ表示。
- scoreは補助情報として表示してよいが、自動確定条件に使用しない。
- Mask overlayは元Layerと同じTransformを使ってCanvas上へ半透明表示する。
- Candidate切替、追加クリック、RetryはDocumentとHistoryを変更しない。

12.5 確定

Bから得た8bit maskとAが保持する元RGBAを使い、`alpha' = round(alpha * mask / 255)`で新Rasterを作る。

生成Layer：

- 元画像と同じpixel寸法
- 元Layer直上
- 元Layerのx/y/rotation/scale/opacityを引継ぎ
- `visible=true`
- 初期名「元レイヤー名（AI抽出）」

- 元Layerは変更しない

`AddLayerCommand`として1回で確定し、Redoでは保存済みLayerを再追加して再推論しない。

12.6 失効条件

Raster画素変更、対象削除、Document切替、AI取消、サービス再起動でAI sessionを失効する。位置・回転・scale・opacity等、元Raster画素が同一の変更では登録画像を維持し、Overlayと確定Layerへ最新Transformを使用する。

---

13. PSD DTOの内部Documentへの登録

13.1 `PsdImportCoordinator`

Version 1ではPSDを「新しいDocumentとして開く」操作を基本とし、既存Documentへmergeする機能は対象外とする。

処理順：

1. Bへ`importPsd(Path)`要求
2. `PsdImportResult`受領
3. A側で内部Layer候補と`PsdImportMetadata`をメモリ上に構築
4. Warningを利用者へ表示
5. 続行ならDocumentを公開、Cancelなら何も変更しない

13.2 Node変換

- `RASTER` / `REPLACEMENT_RASTER`：RasterLayerを生成。
- `GROUP_METADATA` / `METADATA_ONLY` / `EMPTY_METADATA`：描画Layerを生成せずmetadataへ保持。
- Layer順はB DTOのsiblingIndex「下から上0始まり」を基準に平坦な描画順へ正規化する。
- `effectiveVisible`を初期`visible`へ使用。
- Bが描画用に返した`opacity`をそのままLayer opacityへ使用し、`effectiveOpacity`を重ねて適用しない。
- 各生成Layerに`PsdOrigin(importId,nodeId)`を付与する。

13.3 Metadata

```
record PsdOrigin(UUID importId, String nodeId) {}

class PsdImportMetadata {
    UUID importId;
    List<PsdNodeMetadata> originalNodes;
    List<ImportWarning> warnings;
}
```

元階層、kind、自己表示、元opacity、blendMode、importDisposition、Warningを保持する。`.jimg`では`imports/{importId}.json`へ永続化する。

13.4 Warning UI

`RASTERIZED`、`FLATTENED`等で編集性が失われた内容を具体的に表示する。「互換性に問題があります」だけで終わらせない。利用者がCancelした場合は新Documentを公開しない。

---

14. A側テスト設計

JavaテストはJUnit 5を使用する。B Processorそのものの画素正解性はB側試験で確認し、Aは契約・状態遷移・Document反映を中心に検証する。

ID	対象	入力／操作	合格条件
D01	Document	新規、保存、編集、Undo	save pointとdirtyが状態に一致
L01	LayerManager	追加・削除・選択	順序とactiveLayer規則が一致
L02	Layer属性	visible/opacity/name/move/reorder	1操作1Command、Undo/Redo一致
R01	Renderer	透明度・重なり・clip	Layer順とopacityの二重適用なし
T01	座標	Zoom/Pan/位置/回転/scale	Screen→Layer逆変換が期待位

ID	対象	入力／操作	合格条件
			置
T02	Brush/Eraser	1 Stroke	1Undo単位、Cancelで変更なし
H01	History	execute→undo→redo→新規編集	stackとredo破棄が正しい
H02	履歴上限	256MiB超過	古いCommand破棄、単一超過は未適用
H03	Redo失敗	B Stubが失敗	Documentとstackを維持
F01	画像I/O	PNG/JPEG読込・書出	寸法、alpha規則、保存失敗時状態維持
J01	.jimg	複数Layer/Text/PSD metadata	round-tripで状態復元
J02	.jimg異常	未知Version、欠落asset、Traversal	拒否し既存Document変更なし
A01	AI候補	3候補、score順不同選択	任意候補を選択可能、自動確定なし
A02	AI競合	request a後b、a後着	aをUIへ反映しない
A03	AI確定	半透明RGBA＋二値mask	alpha式一致、元Layer不変、Undo/Redo可
P01	PSD登録	RASTER/metadata/Warning	二重描画なし、Cancelで未反映
E01	Error UI	主要ErrorCode	文字列解析なしでUI分岐

14.1 共同結合テスト

B側I01〜I05と同じシナリオを正本とする。

- I01：PNG → B処理 → A表示 → Undo → Redo → 保存
- I02：AI最大3候補 → 補正 → 確定 → Layer → Undo/Redo

- I03：処理中に編集・削除・Document切替・取消し、古い結果を適用しない
- I04：サービス停止・モデルなしでも通常編集・保存を継続
- I05：PSD階層metadata・Warning・通常Raster個別編集・二重描画なし

---

15. 実装順・結合運用・7週間成果物

15.1 7週間のA成果物

週	A成果物と合流条件
1	JavaFX骨格、Document/Layer/History契約、共通contractレビュー
2	PNG/JPEG Open/Export、Canvas、Layer基本、未保存管理。B基本ProcessorをStub/実装で接続
3	Text/Brush/Eraser、Layer UI、History基本。Blur/Mosaic等と結合可能
4	256px TileDiff、AI最大3候補UI、AI生成Layer、Undo/Redo結合
5	`.jimg` save/load、Error UI、MUST受入フロー
6	SHOULD、PSD UI、回帰修正、UI polish
7	提出版固定、操作手順、デモ、試験記録

15.2 Git運用

```
main          : demo可能な安定版
integration   : A/B結合確認
feature/a-*   : Aの短期branch
feature/b-*   : Bの短期branch
```

- 1機能1PRを基本とする。
- `common-contract`、build、依存、API、DTO変更は相互review必須。
- 相手所有moduleを変更する前に相談する。
- MUST中は無関係な大規模refactorを避ける。

15.3 定例結合

毎週月曜日 16:00〜を定例の結合作業候補とする。

- GitHub上のA/B最新成果物を`integration`へ統合。

- build、自動test、当週の結合scenarioを実行。
- 問題を記録し、A/Bどちらの責務かを切り分ける。

問題がある場合は、**翌火曜日 12:45〜14:15の授業内**で共有し、contract変更の要否、修正担当、次回までの作業を決める。月曜に結合できなかった場合も授業内で状況共有する。

15.4 遅延時

- Bが遅れていてもAはStubを使ってUI/Document/Historyを進める。
- 第4週末でAI未統合ならPSD SHOULDよりAI MUSTを優先する。
- 第5週末でMUST受入未達なら新機能追加を止め、両者で結合・不具合修正へ集中する。
- ADVANCEDはMUST受入が安定してから着手する。

---

16. 要件・合意事項との対応と実装記録

対象	本書	主な確認
FR-01〜04	4, 9, 10	F01, J01, J02
FR-08〜10	5〜7, 9	T02, H01
FR-16〜24	5, 6, 9	L01, L02, R01
FR-05〜07 / 11〜15	8, 11	I01, H03
FR-29〜34	4, 7, 12	A01〜A03, I02〜I04
FR-25〜27	10, 13	P01, I05
FR-35〜36	9	H01〜H03
FR-37〜38	3, 6, 7	T01
AG-01〜09	全章	共通contract・世代・AI・Canvas・PSD・Preview・History・環境・結合

16.1 実装時に残す記録

- 合意済みcommon-contract版
- B側algorithmVersionとHTTP API版
- Java / JavaFX / Gradle / Jacksonの固定版

- `.jimg` schema version
- Unit / Integration test結果
- 結合時に発生したIssueと責務判断
- 4K等の性能確認時の端末条件
- 提出版Git tag / commit hash

✅ A側実装では「Bの結果を受け取ったらすぐDocumentへ書く」のではなく、**対象ID・contentRevision・取消状態・AIではsession/requestIdまで再検証し、成功結果だけをCommandとして一括反映する**ことを最重要の統合規則とする。